

Quadcopter Control Algorithm: A Fuzzy-Neural Network Control Approach

Submitted as part of the requirements for:
CE902 Professional Practice and Research Methodology

Author: Jiwei Chen

Supervisor: Dr. Michael Fairbank

Abstract

This project aims to build an Adaptive Neuro-Fuzzy Inference System (ANFIS) for quadcopter control. By combining neural networks and fuzzy logic, the approach is expected to handle nonlinear dynamics and disturbances for enhanced trajectory tracking and disturbance rejection. Simulation and real-world experiments validate significantly improved stability, accuracy, and robustness over traditional PID controllers.

Keywords: quadcopter, neural networks, fuzzy control, drone, PID controller

Contents

1	Introduction	1
2	Literature Review	2
2.1	Proportional-Integral-Derivative Controllers	2
2.2	Limitations of Traditional PID Controllers	3
2.3	Enhanced Control Algorithms for Quadcopters	3
3	Methodology	4
3.1	Mathematical models of the quadcopter	4
3.2	The neural network architecture	6
3.2.1	Input feature	6
3.2.2	Output feature	7
3.2.3	The Adaptive Neuro-Fuzzy Inference System (ANFIS) Structure . .	7
4	Implementation	10
4.1	Evaluation Plan	11
4.1.1	Quantitative Performance	11
4.1.2	Test Scenarios	11
4.1.3	Validation Methodology	12
4.2	Expected Results	12
4.2.1	Quantitative Performance	12
4.2.2	Test Scenario Performance	13
5	Project Management	13
5.1	Project Methodology	13
5.2	Project Timeline and Development Phases	14
5.3	Resource Allocation	15
5.4	Risk Management	15
5.5	Quality Assurance	16
A	Quadcopter Model	17
B	Kanban Board	18
C	Gantt Chart	18

1 Introduction

Quadcopters, commonly known as drones, have become essential in various industries such as logistics, emergency response, and environmental monitoring. Despite their numerous applications, quadcopters face significant challenges including limited payload, high operational costs, complex nonlinear dynamics, and susceptibility to environmental disturbances. These issues underscore the necessity for advanced control strategies that can ensure robust and reliable performance under diverse conditions.

The primary objective of this project is to develop a quadcopter control system based on an Adaptive Neuro-Fuzzy Inference System (ANFIS). By integrating the adaptive learning capabilities of neural networks with the interpretability of fuzzy logic, the ANFIS controller is designed to handle the nonlinearities and uncertainties of quadcopter dynamics, delivering enhanced stability, precision, and responsiveness when compared to PID controller.

The overall structure of this document is organized as follows:

- **Literature Review:** Reviews existing control methodologies for quadcopters, examines the limitations of conventional PID controllers, and highlights the potential benefits of fuzzy-neural network approaches.
- **Methodology:** Describes the underlying mathematical models of quadcopter dynamics, details the architecture of the ANFIS controller, and explains the procedures for network training and parameter optimization.
- **Implementation:** Outlines the practical aspects of the system development, including software and hardware configurations, simulation setups, and integration challenges.
- **Evaluation Plan:** Establishes the criteria and methods for performance evaluation through both simulation tests and real-world experiments, covering aspects such as steady-state error, transient response, and disturbance rejection.
- **Project Management:** Discusses the project timeline, resource management, risk mitigation strategies, and quality assurance measures to ensure successful project execution.

2 Literature Review

This section will explore the literature on quadcopter control systems, highlighting existing approaches and their limitations. The review will inform the proposed methodology by identifying gaps in current research.

2.1 Proportional-Integral-Derivative Controllers

Proportional-Integral-Derivative (PID) controllers are the most common control systems used in quadcopters. PID controllers operate by continuously calculating an error value as the difference between a desired setpoint and a measured process variable. They then apply a correction based on proportional, integral, and derivative terms. Specifically, the proportional component multiplies the current tracking error (difference between desired output and actual output) by a proportional gain to determine the control output, thus quickly driving the system to the desired setpoint. The proportional term is represented as:

$$P(t) = K_p e(t) \quad (1)$$

where $P(t)$ is the proportional term, K_p is the proportional gain, and $e(t)$ is the error at time t .

The integral component accumulates the error over time and multiplies the sum of these errors by an integral gain, and ensures that the steady-state error approaches zero. The integral term is represented as:

$$I(t) = K_i \int_0^t e(\tau) d\tau \quad (2)$$

where $I(t)$ is the integral term, K_i is the integral gain, and $e(\tau)$ is the error at time τ .

The derivative component predicts the future behavior of the error based on its rate of change, reducing the overshoot and settling time. The derivative term is represented as:

$$D(t) = K_d \frac{de(t)}{dt} \quad (3)$$

where $D(t)$ is the derivative term, K_d is the derivative gain, and $\frac{de(t)}{dt}$ is the derivative of the error.

Combining these three terms, the complete PID controller at time t is:

$$u(t) = P(t) + I(t) + D(t) \quad (4)$$

Hagras (nd) pointed out that in practical applications, since it can be challenging or costly to precisely derive or model the system dynamics, PID parameters are often tuned manually through a trial-and-error process. Typically, the tuning process starts with a small proportional gain, slowly adjusting the derivative gain to reduce swing, and then tuning the integral gain to eliminate steady-state errors.

2.2 Limitations of Traditional PID Controllers

Despite their widespread use, traditional PID controllers have serious limitations when used with quadcopter systems. PID controllers are basically linear and low-bandwidth control systems, which makes them not well suited for highly nonlinear systems that face strong, frequent disturbances—problems that are common with quadcopters, especially when flying different tasks in changing environments. These limitations highlight the need for more advanced control methods for quadcopter systems (Jiang et al., 2020).

2.3 Enhanced Control Algorithms for Quadcopters

Jiang et al. (2020) proposes and evaluates an adaptive control system for quadrotor UAVs by integrating a **sigma-pi neural network (SPNN)** with dynamic inversion and PID control methods. They developed a neural-network-based compensator to address model uncertainties, payload variations, and external disturbances typically encountered in UAV operations. Through both simulation and real-world experiments, the SPNN-based system demonstrated significant improvements in reducing inversion errors, trajectory deviations, and external disturbances compared to conventional PID-based controllers.

Both El Hamidi et al. (2019) and Santoso et al. (2016) investigate novel control methodologies for quadcopters, leveraging fuzzy logic and neural network approaches to address the nonlinear dynamics of quadcopters. Santoso et al. (2016) employs **Adaptive NeuroFuzzy Inference Systems (ANFIS)** to construct comprehensive models of quadcopter motion. This approach simulates dynamic responses along the roll axis, pitch axis, and thrust axis, by integrating fuzzy neural network, thereby capturing complex flight behaviors with greater precision. The primary advantage of this methodology lies in its adaptability and robust response to environmental uncertainties.

In a similar research direction, El Hamidi et al. (2019) examines enhancement strategies for quadcopter stability and trajectory control through the application of neural networks and fuzzy logic frameworks to optimize conventional PID or PD controllers. Their investigation demonstrates how neural networks can dynamically adjust control parame-

ters through iterative learning processes, while fuzzy logic controllers further refine performance metrics with assistance from Particle Swarm Optimization (PSO) techniques. This integration yields control systems exhibiting superior stability, precision, and trajectory following capabilities.

Both research teams acknowledge the inherent limitations of traditional linear control methodologies when confronting stochastic disturbances and nonlinear behaviors characteristic of quadcopter systems.

3 Methodology

This section describes the proposed methods, algorithms, models, and approaches that will be applied in this project. The methodology will evolve as the project progresses, and adjustments will be made based on initial findings.

3.1 Mathematical models of the quadcopter

Accurate mathematical modeling of quadcopter dynamics is essential for developing effective control algorithms. This section presents the fundamental kinematic equations that describe the quadcopter's motion in six degrees of freedom, drawing upon established models in the literature, particularly the work of Oktay and Köse (2019) and Praveen et al. (2016). The following contents inform the approach will be taken to model the quadcopter dynamics.:

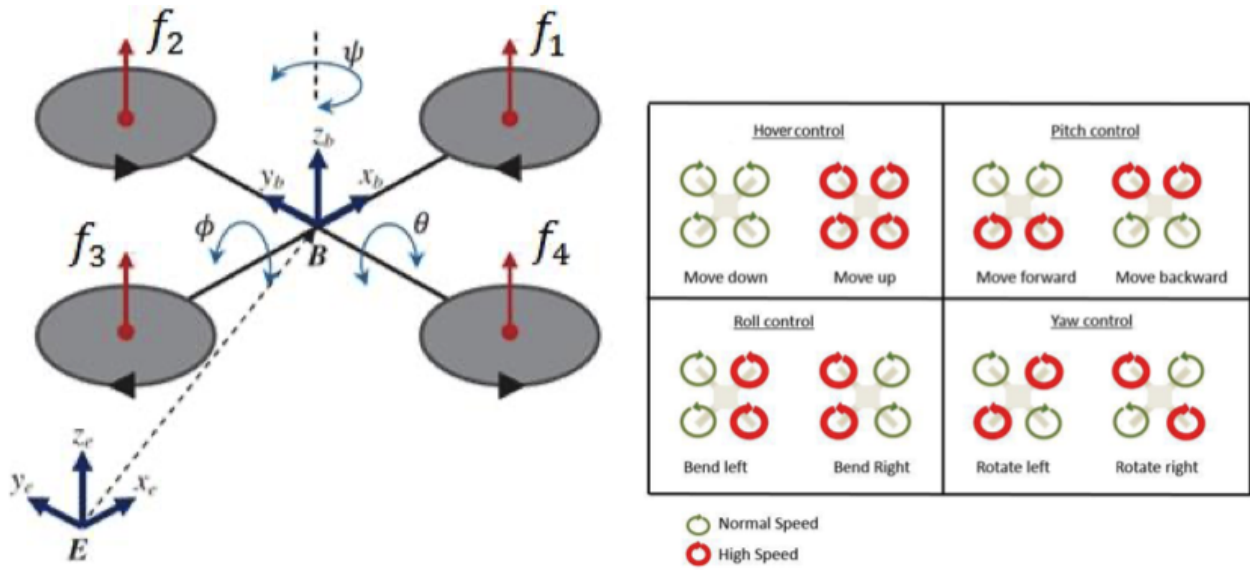
$$\begin{cases} \ddot{x} = (\sin \psi \sin \phi + \cos \psi \sin \theta \cos \phi) \frac{u_1}{m} \\ \ddot{y} = (-\cos \psi \sin \phi + \sin \psi \sin \theta \cos \phi) \frac{u_1}{m} \\ \ddot{z} = -g + (\cos \theta \cos \phi) \frac{u_1}{m} \end{cases} \quad (5)$$

$$\begin{cases} \ddot{\phi} = \frac{I_y - I_z}{I_x} \dot{\theta} \dot{\psi} + \frac{l}{I_x} u_2 \\ \ddot{\theta} = \frac{I_z - I_x}{I_y} \dot{\phi} \dot{\psi} + \frac{l}{I_y} u_3 \\ \ddot{\psi} = \frac{I_x - I_y}{I_z} \dot{\phi} \dot{\theta} + \frac{1}{I_z} u_4 \end{cases} \quad (6)$$

where:

- (x, y, z) are the position
- $(\dot{x}, \dot{y}, \dot{z})$ are the linear velocities,
- $(\ddot{x}, \ddot{y}, \ddot{z})$ are the linear accelerations,

- (ϕ, θ, ψ) are the roll, pitch, and yaw angles,
- $(\dot{\phi}, \dot{\theta}, \dot{\psi})$ are the angular velocities,
- $(\ddot{\phi}, \ddot{\theta}, \ddot{\psi})$ are the angular accelerations,
- (I_x, I_y, I_z) are the moments of inertia,
- (u_1, u_2, u_3, u_4) are the control inputs.
- m is the mass, g is the acceleration due to gravity. l is the distance from the center of mass to the motors.



(a) Quadcopter configuration. Source: Oktay and Köse (2019)

(b) Quadcopter movement. Source: Oktay and Köse (2019)

Figure 1

3.2 The neural network architecture

The neural network architecture designed for this project draws upon significant research in quadcopter control systems. Building on the adaptive sigma-pi neural network approach proposed by Jiang et al. (2020), which effectively addresses model uncertainties and external disturbances, the architecture incorporates key elements from recent advances in the field. Additionally, the end-to-end neural network control methodology presented by Ferede et al. (2024) informs the design choices regarding network topology and training procedures. This foundation enables the development of a neural network architecture specifically tailored to the unique challenges of quadcopter control in complex environments.

3.2.1 Input feature

Attitude Errors: The difference between the desired and actual altitude of the quadcopter.

$$\begin{aligned}e_{\phi} &= \phi_{desired} - \phi_{actual} \\e_{\theta} &= \theta_{desired} - \theta_{actual} \\e_{\psi} &= \psi_{desired} - \psi_{actual} \\e_z &= z_{desired} - z_{actual}\end{aligned}\tag{7}$$

Angular velocity errors: The difference between the desired and actual angular velocities of the quadcopter.

$$\begin{aligned}e_{\dot{\phi}} &= \dot{\phi}_{desired} - \dot{\phi}_{actual} \\e_{\dot{\theta}} &= \dot{\theta}_{desired} - \dot{\theta}_{actual} \\e_{\dot{\psi}} &= \dot{\psi}_{desired} - \dot{\psi}_{actual}\end{aligned}\tag{8}$$

position and acceleration error: The difference between the desired and actual z-axis position and acceleration of the quadcopter.

$$\begin{aligned}e_x &= x_{desired} - x_{actual} \\e_{\dot{x}} &= \dot{x}_{desired} - \dot{x}_{actual} \\e_y &= y_{desired} - y_{actual} \\e_{\dot{y}} &= \dot{y}_{desired} - \dot{y}_{actual} \\e_z &= z_{desired} - z_{actual} \\e_{\ddot{z}} &= \ddot{z}_{desired} - \ddot{z}_{actual}\end{aligned}\tag{9}$$

3.2.2 Output feature

The control inputs to the quadcopter motors are the output features of the neural network.

$$\begin{aligned}u_1 &= m(\ddot{x} + g) \\u_2 &= I_x \ddot{\phi} \\u_3 &= I_y \ddot{\theta} \\u_4 &= I_z \ddot{\psi}\end{aligned}\tag{10}$$

Building upon the input and output features defined above, this project aims to implement an Adaptive Neuro-Fuzzy Inference System (ANFIS) approach to quadcopter control. ANFIS combines the interpretability of fuzzy logic systems with the adaptive learning capabilities of neural networks, creating a hybrid architecture particularly suited for nonlinear control problems (Jang, 1993) like quadcopter dynamics. This integration allows the controller to handle the inherent nonlinearities in quadcopter behavior while adapting to changing flight conditions and disturbances. By processing the error signals through a structured neural network with fuzzy inference capabilities, the system can generate appropriate control signals that maintain stability across diverse operating conditions. The following section details the specific ANFIS architecture designed for this project.

3.2.3 The Adaptive Neuro-Fuzzy Inference System (ANFIS) Structure

The ANFIS controller employs a classic five-layer network architecture adapted from Karaboga and Kaya (2019) to optimized specifically for quadcopter control:

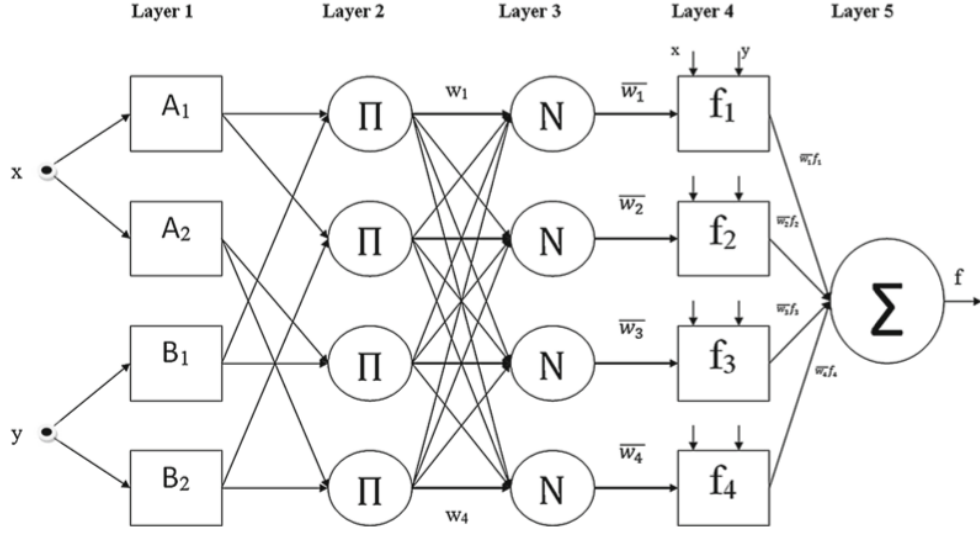


Figure 2: An example of ANFIS with two inputs and one output. Source: Karaboga and Kaya (2019)

- **Input:** Processes multiple input features representing state errors and environmental information:
 - **Primary state errors:** Attitude errors (roll, pitch, yaw), angular velocity errors, and position errors as defined in equations 7-9, totaling 12 state error inputs.
 - **Obstacle proximity (if available):** Distance measurements from sensors (e.g., LiDAR, or depth cameras) that detect obstacles in the flight path, triggering avoidance maneuvers when objects are within critical proximity.
- **Layer 1 (Fuzzification Layer):** Transforms continuous inputs into linguistic variables through membership functions. Each input variable utilizes 3-5 Gaussian membership functions (such as "large/medium/small" errors or "near/far" distances) that form the basis for fuzzy rule construction.

$$O_i^{(1)} = \mu_{A_i}(x_1), \quad O_j^{(1)} = \mu_{B_j}(x_2), \dots \quad (11)$$

where x_1 and x_2 are the inputs, μ_{A_i} and μ_{B_j} are Gaussian membership functions, $i = 1, 2, \dots, n$ and $j = 1, 2, \dots, m$.

- **Layer 2 (Rule Layer):** Implements the fuzzy rule base. Rule nodes combine input membership values using logical operators (e.g., AND, OR) to determine rule firing strengths.

$$O_{ij}^{(2)} = w_{ij} = \mu_{A_i}(x_1) \times \mu_{B_j}(x_2) \quad (12)$$

where w_{ij} is the firing strength of the rule connecting the i -th membership function of input x_1 and the j -th membership function of input x_2 .

- **Layer 3 (Normalization):** The normalized firing strength value is calculated as follows:

$$O_{ij}^{(3)} = \bar{w}_{ij} = \frac{w_{ij}}{\sum_{i=1}^n \sum_{j=1}^m w_{ij}} \quad (13)$$

- **Layer 4 (Defuzzification Layer):** Aggregates the weighted rule outputs to produce the final control signals for the quadcopter motors.

$$O_{ij}^{(4)} = \bar{w}_{ij} f_{ij} = \bar{w}_{ij} (p_{ij} x_1 + q_{ij} x_2 + r_{ij}) \quad (14)$$

where p_{ij} , q_{ij} , and r_{ij} are consequent parameters of the first-order Sugeno fuzzy model.

- **Layer 5 (Output Layer):** Produces the four motor commands that correspond to quadcopter thrust forces.

$$O^{(5)} = \sum_{i=1}^n \sum_{j=1}^m \bar{w}_{ij} f_{ij} = \frac{\sum_{i=1}^n \sum_{j=1}^m w_{ij} f_{ij}}{\sum_{i=1}^n \sum_{j=1}^m w_{ij}} \quad (15)$$

which is the single output of the ANFIS system, calculated as the weighted sum of all rule outputs.

The network parameters are optimized through **supervised learning** using labeled input-output pairs generated from trajectories and corresponding control signals collect from mathematical models. The following outlines the principal training methodology for the neural aspect of ANFIS:

- Uses **least squares estimation (LSE)** in the forward pass to optimize the consequent parameters in Layers 3-4.
- Applies **gradient descent via backpropagation** in the backward pass to tune the premise parameters in Layer 1 (the Gaussian membership function centers and widths)
- The loss function for training **mean squared error (MSE)** expressed as follows:

$$J = \frac{1}{2N} \sum_{k=1}^N |\mathbf{u}_k^* - \mathbf{u}_k|^2 \quad (16)$$

where u_k^* represents ideal control outputs, u_k is the ANFIS output.

To reduce dimensionality and rule complexity, the ANFIS structure will be decomposed into separate modules for attitude and altitude control, allowing for more efficient training and real-time computation.

4 Implementation

This section outlines the practical implementation strategy for the neural network and fuzzy control system. It includes software and hardware requirements, and addresses potential technical challenges that may arise during development. The implementation will proceed in phases, beginning with mathematical modeling, followed by simulation development, and concluding with practical testing.

- *Software selection:* MATLAB/Simulink, Python libraries
- *Hardware specifications:*
 - Quadcopter platform
 - Onboard sensors
 - Computational resources
- *Data to monitor and record:*
 - Quadcopters parameters: mass, inertia, motor characteristics.
 - Flight data: position, velocity, acceleration, orientation of the quadcopter.
 - Sensor readings: data from onboard sensors - accelerometers, gyroscopes, LIDAR.
 - Actuator outputs: control signals sent to the motors and their corresponding responses.
 - System state variables: battery level, motor temperatures, and computational load.
- *Testing environments:* MATLAB/Simulink, CSEE Robotic Arena
- *Potential technical challenges:*
 - Integration of neural networks and fuzzy logic control may lead to computational complexity.

- Real-time performance issues due to high computational demands.
- Limited availability of high-quality training data for neural networks.
- Sensor noise and inaccuracies affecting control performance.
- Hardware limitations of the quadcopter platform.
- Ensuring safety during real-world testing.

4.1 Evaluation Plan

This section outlines the evaluation criteria and metrics that will be used to assess the performance of the hybrid control system. It includes both quantitative and qualitative measures to provide a comprehensive evaluation framework.

4.1.1 Quantitative Performance

- **Steady-state errors:** RMS errors in position and attitude during hovering
- **Transient response:** Rise time, settling time, and overshoot during step responses
- **Trajectory tracking accuracy:**
 - Mean absolute error (MAE) between desired and actual trajectories
 - Maximum difference from planned path
 - Path completion time
- **Disturbance rejection:**
 - Recovery time following disturbance
 - Control effort (measured by motor command variance)

4.1.2 Test Scenarios

The controllers will be evaluated across progressively challenging scenarios:

- **Basic flight maneuvers:**
 - Hovering at fixed position
 - Step changes in altitude and position
 - Uniform velocity trajectory following

- **Complex flight patterns:**
 - Figure-eight trajectories with varying speeds
 - Trajectory with sharp turns and sudden stops.
 - Continuous altitude changes with varying rates
- **Disturbance testing:**
 - Simulated wind gusts of varying intensities.
 - Sudden payload changes.
- **Obstacle avoidance scenarios:**
 - Static obstacle avoidance.
 - Avoidance of suddenly appearing obstacles.

4.1.3 Validation Methodology

The evaluation will follow a two-stage validation approach:

- **Simulation testing:** Using MATLAB/Simulink environment with high-fidelity quadcopter models that incorporate realistic aerodynamics, sensor noise, and environmental disturbances
- **Physical testing:** Implementation on actual quadcopter platform in the CSEE Robotic Arena, with controlled testing conditions to ensure repeatability and safety

4.2 Expected Results

Based on the evaluation plan, this project anticipates the following outcomes:

4.2.1 Quantitative Performance

- **Steady-state errors:** Lower errors compared to traditional PID controllers.
- **Transient response:** Improved rise time, reduced settling time, and minimized overshoot, which resulting in smoother transitions between flight states.
- **Trajectory tracking accuracy:** Achieve path following with reduced mean absolute errors and smaller maximum deviations from planned trajectories.

- **Disturbance rejection:** Enhanced recovery capabilities following external disturbances, with faster stabilization times.

4.2.2 Test Scenario Performance

- **Basic flight maneuvers:** Superior stability when hovering and give precise responses to step changes in position and altitude.
- **Complex flight patterns:** Be able to maintain tracking accuracy while preserving smooth motion.
- **Disturbance handling:** Superior adaptation to both simulated wind gusts and payload variations is, with minimal deviation from intended flight paths.
- **Obstacle avoidance:** Effective navigation around both static and suddenly appearing obstacles while maintaining flight stability.

These outcomes would validate the effectiveness of integrating neural networks with fuzzy control for quadcopter systems and demonstrate the practical advantages of this hybrid approach over traditional control methods.

5 Project Management

This section details the project management approach for developing and testing the fuzzy-neural network quadcopter control system. The project will follow a structured methodology with clearly defined phases, milestones, and risk mitigation strategies.

5.1 Project Methodology

The project employs the Agile methodology for development and implementation. Agile development is recognized for its iterative approach, flexibility to accommodate changes, and emphasis on delivering functional components in brief cycles. This methodology is particularly appropriate for the quadcopter control algorithm project, enabling progressive development and enhancement of the algorithm in accordance with technical requirements and testing feedback. The iterative nature of Agile aligns well with the incremental complexity approach planned for the controller implementation.

5.2 Project Timeline and Development Phases

The project is structured into four main phases, with specific activities and milestones:

- **Phase 1: Mathematical Modeling and System Identification** (2025/05/04 - 2025/05/17)
 - Literature review completion.
 - Integration of open-source quadcopter mathematical model.
 - Parameter adjustment and system identification.
 - Implementation of existing PID controller for baseline comparison.
 - **Milestone:** Achieve a validated mathematical model with accurate dynamics.
- **Phase 2: Simulation Development and Testing** (2025/05/18 - 2025/07/05)
 - Implement ANFIS controller design.
 - Development of the training procedure for neural-fuzzy parameters.
 - Conduct initial simulation testing under ideal conditions.
 - Introduce disturbance factors.
 - Perform comparative analysis with PID controllers.
 - **Milestone:** Functional controller with performance metrics exceeding baseline in simulation.
- **Phase 3: Real-world Implementation and Evaluation** (2025/07/06 - 2025/08/16)
 - Integrate hardware and configure the platform.
 - Adapt the controller for real-time operation.
 - Conduct basic flight maneuver testing in a controlled environment.
 - Progressive testing of complex scenarios as outlined in the evaluation plan.
 - Data collection and final performance analysis.
 - **Milestone:** Successful demonstration of the controller in physical testing.
- **Phase 4: Final Documentation** (2025/08/17 - 2025/08/24)
 - Finish Report writing and documentation finalization

5.3 Resource Allocation

- **Software resources:**

- MATLAB/Simulink for mathematical modeling and initial simulations
- Python with TensorFlow/PyTorch for neural network implementation
- Version control system (GitLab) for code management
- Tuleap for project management and collaboration

- **Hardware resources:**

- Quadcopter platform with compatible firmware
- Onboard sensors and processing unit
- CSEE Robotic Arena for physical testing

5.4 Risk Management

Key risks identified for this project include:

- High computational complexity of ANFIS controller exceeding real-time capabilities
Mitigation: Modular design approach with potential for simplified versions that maintain core functionality
- Insufficient training data quality affecting neural network performance
Mitigation: Data augmentation techniques and transfer learning from simulation environments
- Hardware failures during physical testing
Mitigation: Conduct hardware-in-the-loop tests and progressive real-world validations to identify and calibrate discrepancies between the simulation model and the actual hardware.
- Delays in complex algorithm implementation
Mitigation: Prioritize essential functionality and incrementally integrate advanced features.
- Limited access to testing facilities
Mitigation: Early booking of robotic arena.

5.5 Quality Assurance

To ensure the quality of research and development:

- Regular code reviews and validation against mathematical principles
- Comprehensive documentation of design decisions and experimental results
- Comparison against baseline implementations to quantify improvements
- Validation through both simulation and physical testing environments

More details on project management are provided in the Appendix.

References

- Ferede, R., de Croon, G., De Wagter, C., and Izzo, D. (2024). End-to-end neural network based optimal quadcopter control. *Robotics and Autonomous Systems*, 172:104588.
- Hagras, H. (n.d.). Robot control. Lecture slides, University of Essex. Course: CE801-Intelligent Systems and Robotics.
- Jang, J.-S. (1993). Anfis: adaptive-network-based fuzzy inference system. *IEEE Transactions on Systems, Man, and Cybernetics*, 23(3):665–685.
- Jiang, F., Pourpanah, F., and Hao, Q. (2020). Design, implementation, and evaluation of a neural-network-based quadcopter uav system. *IEEE Transactions on Industrial Electronics*, 67(3):2076–2085.
- Karaboga, D. and Kaya, E. (2019). Adaptive network based fuzzy inference system (anfis) training approaches: a comprehensive survey. *Artificial Intelligence Review*, 52:2263–2293.
- Oktaç, T. and Köse, O. (2019). Dynamic modeling and simulation of quadcopter for several flight conditions. *Avrupa Bilim ve Teknoloji Dergisi*, (15):132–142.
- Praveen, V., Pillai, S., et al. (2016). Modeling and simulation of quadcopter using pid controller. *International Journal of Control Theory and Applications*, 9(15):7151–7158.

A Quadcopter Model

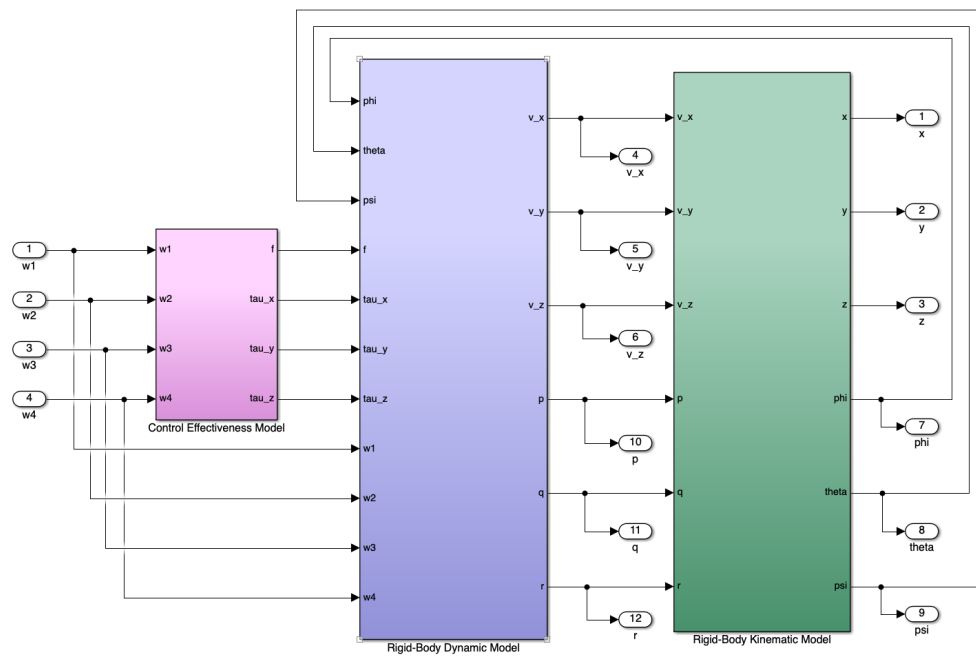
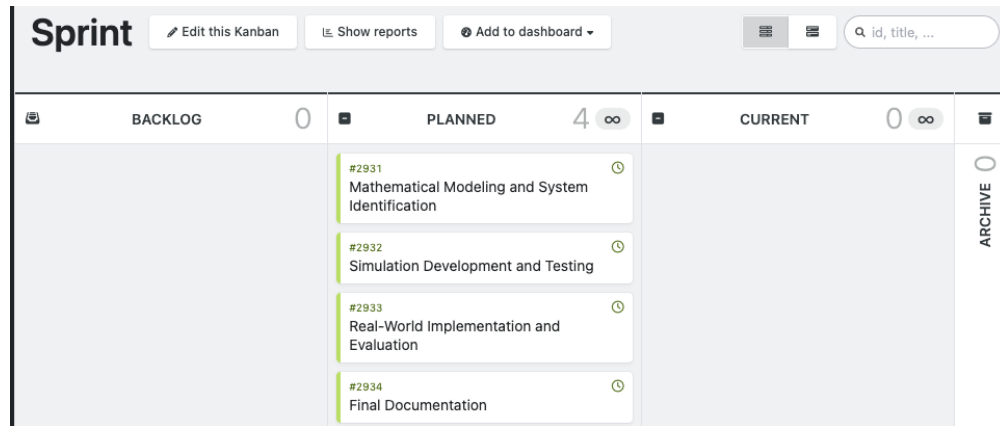
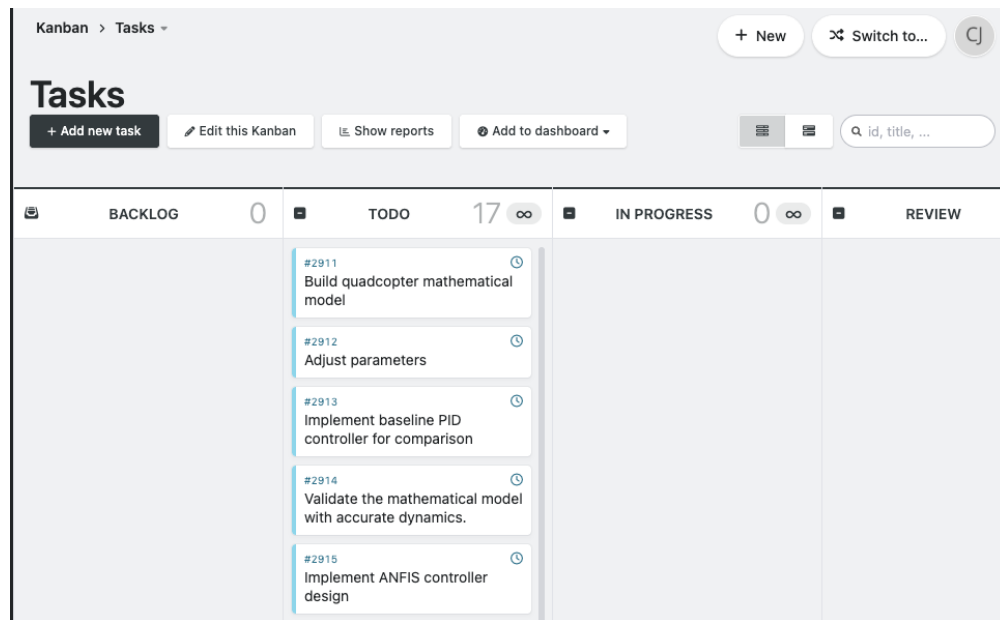


Figure 3: Quadcopter model (not fully functional)

B Kanban Board



(a) Sprints overview



(b) Tasks list

Figure 4: kanban board

C Gantt Chart

[illegible]